

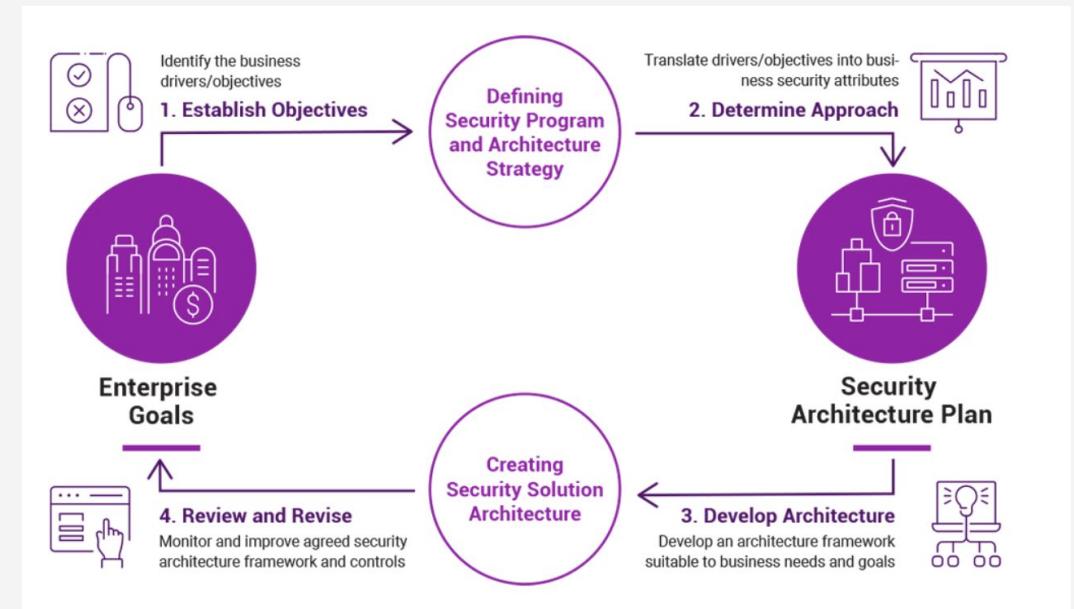
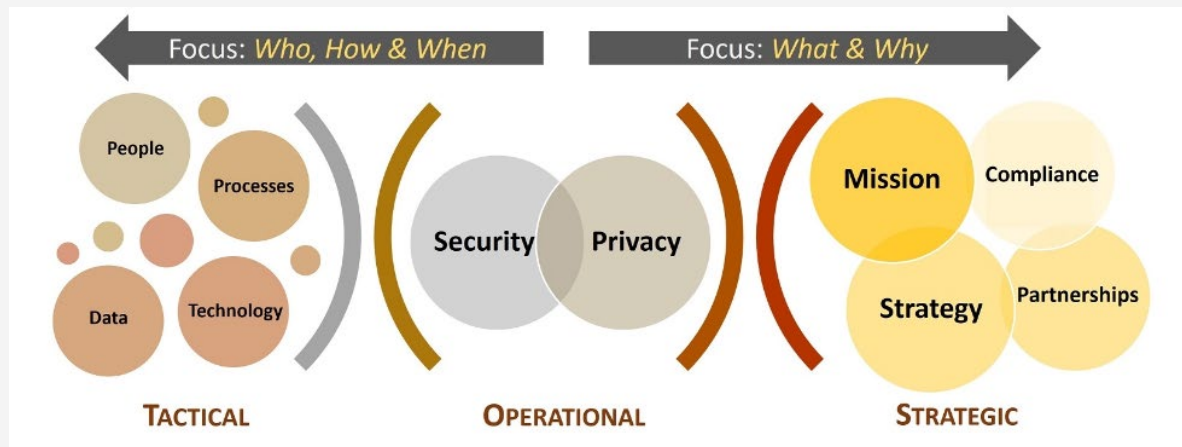


Master Digital Transformation Cyber Security - Aspects of Operational Security

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
(Infrastructure & Security)



Security is a company-wide effort





Security Organisation in Companies

- CISO
- Trust & Assurance
- Identity and Access Management
- Product and Application Security
- Security Operations Center

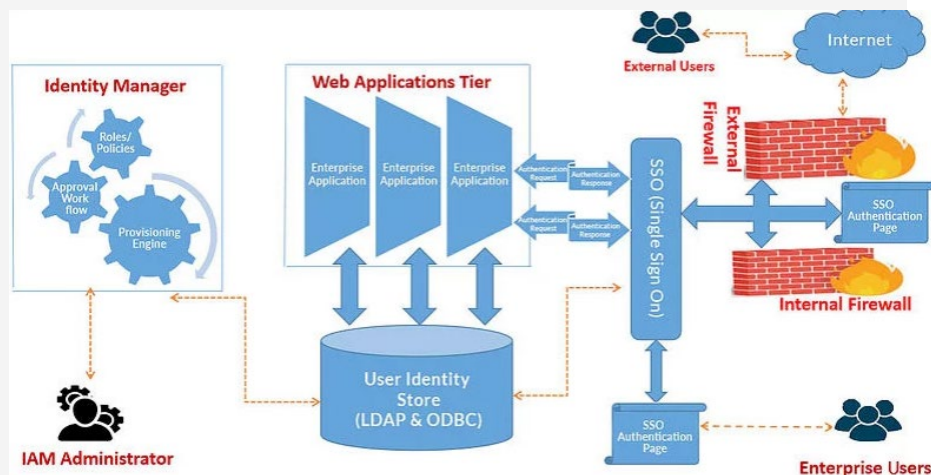
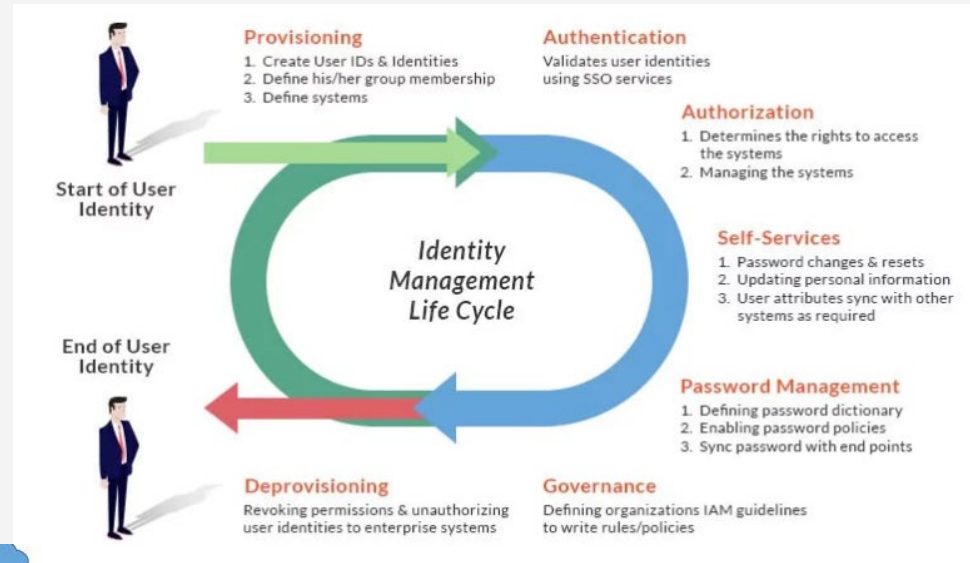
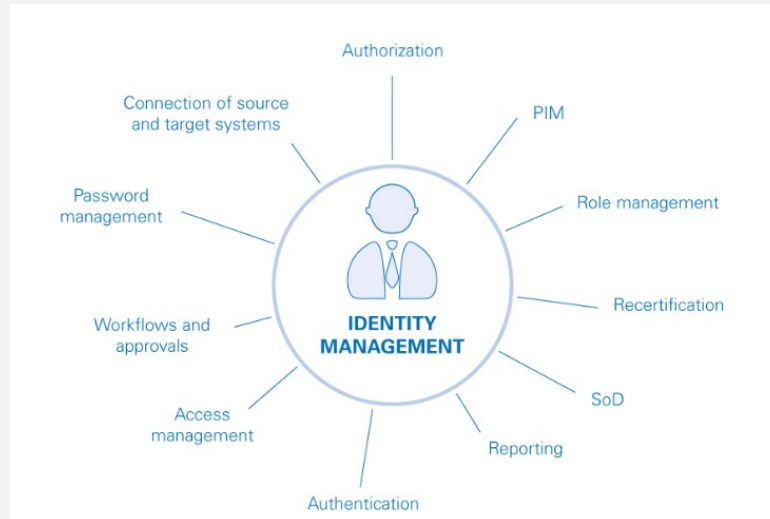
Typical Roles

- CISO
- Security Manager
- Incident Responder und Intrusion Detection Specialists
- Forensics Analyst
- Penetration Tester / Ethical Hacker
- IAM Engineer
- Security Architect
- Security Engineer
- Software Engineer
- Principal Security Engineer
- opt. Crypto Experts
- op. Machine Learning Engineers
- Compliance Manager
- Auditors





Identity and Access Management

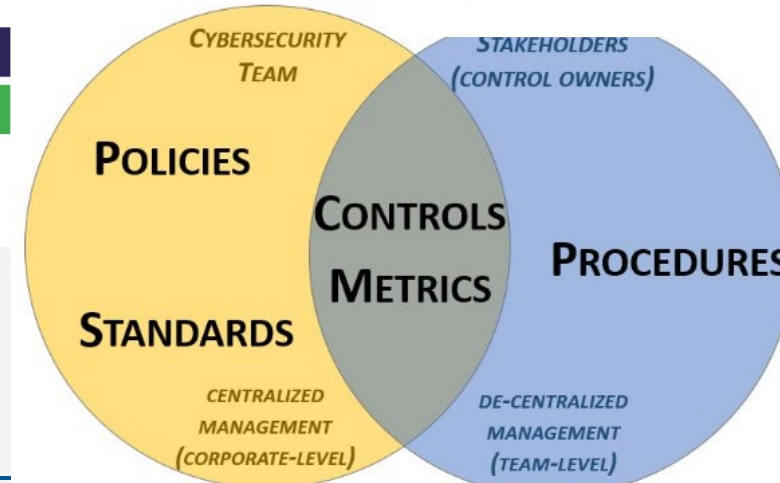
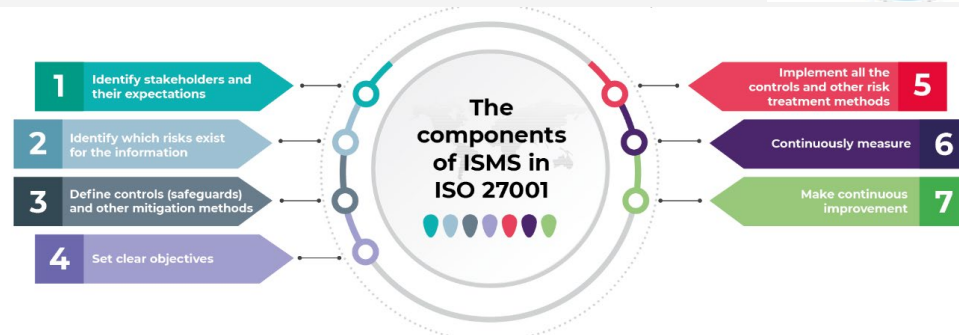
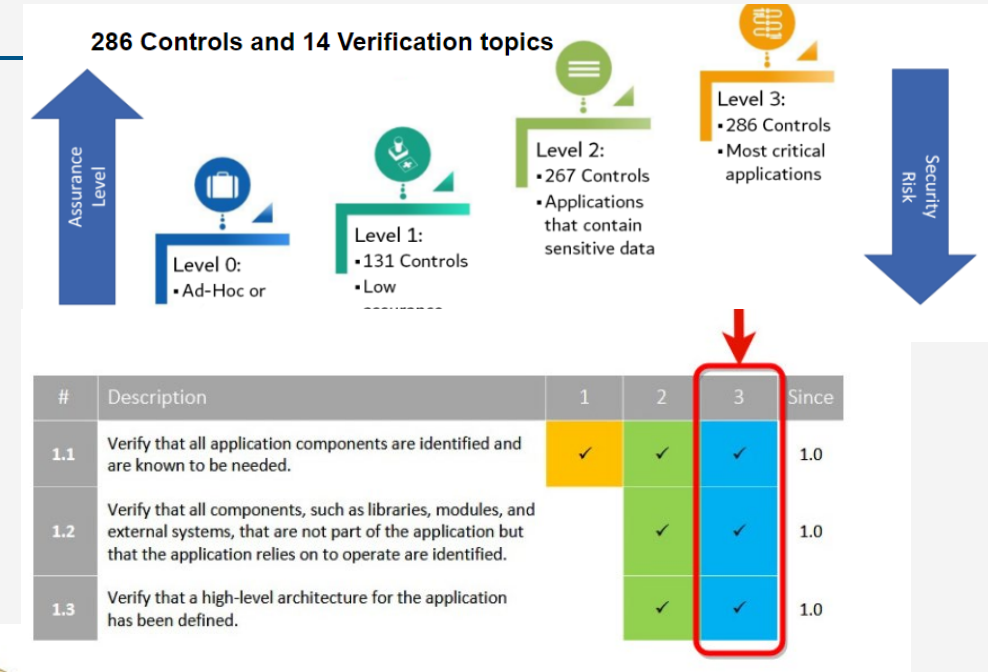
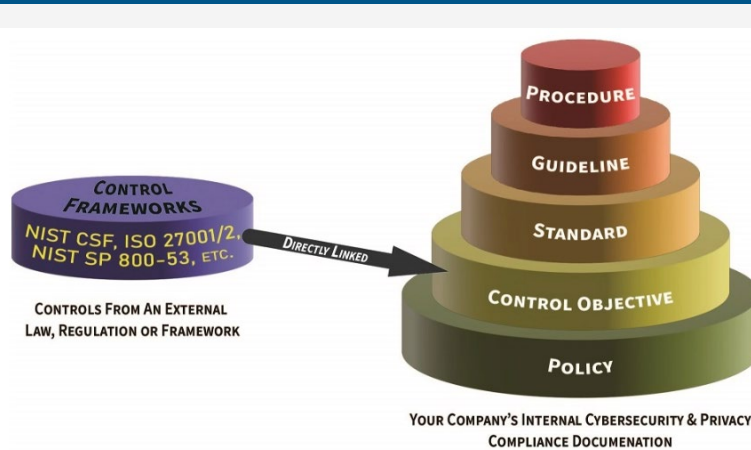


- Most of the fancy crypto part lies here
- A lot of thought goes into design of a manageable authorization system
- Engineering-intense department

The Policy and Management Side - Trust & Assurance



Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

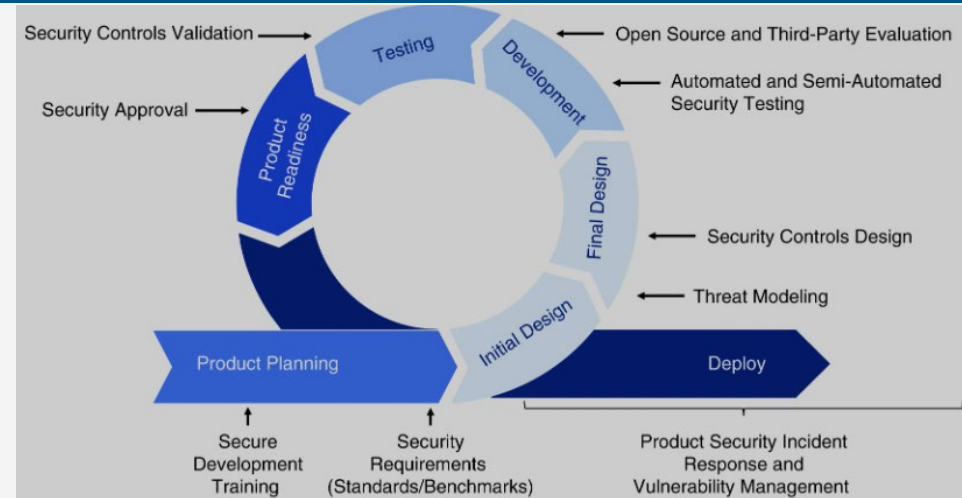
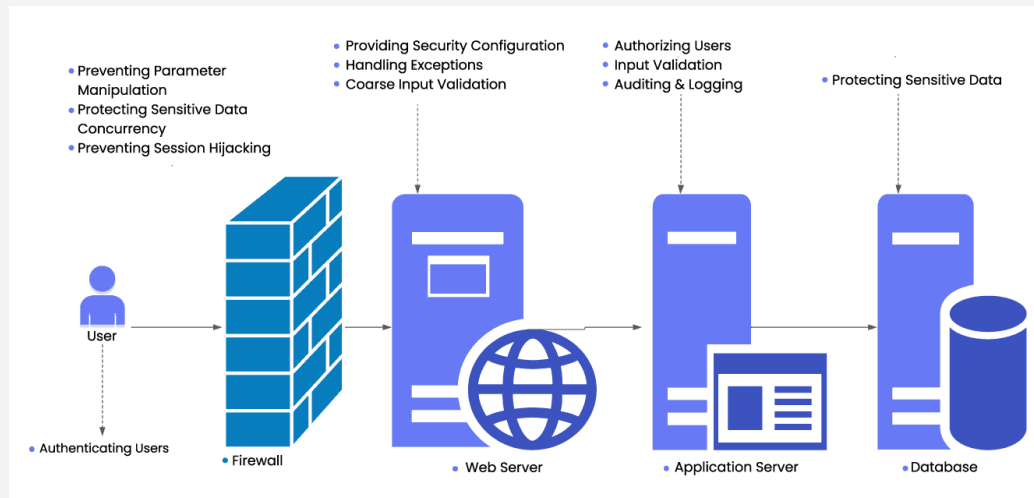


- Tailor standards to own organization
- Measure fulfillment / compliance to those
- Feed requirements from both business and tech into improved requirements

A key component for Shift Left: Product and Application Security



Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

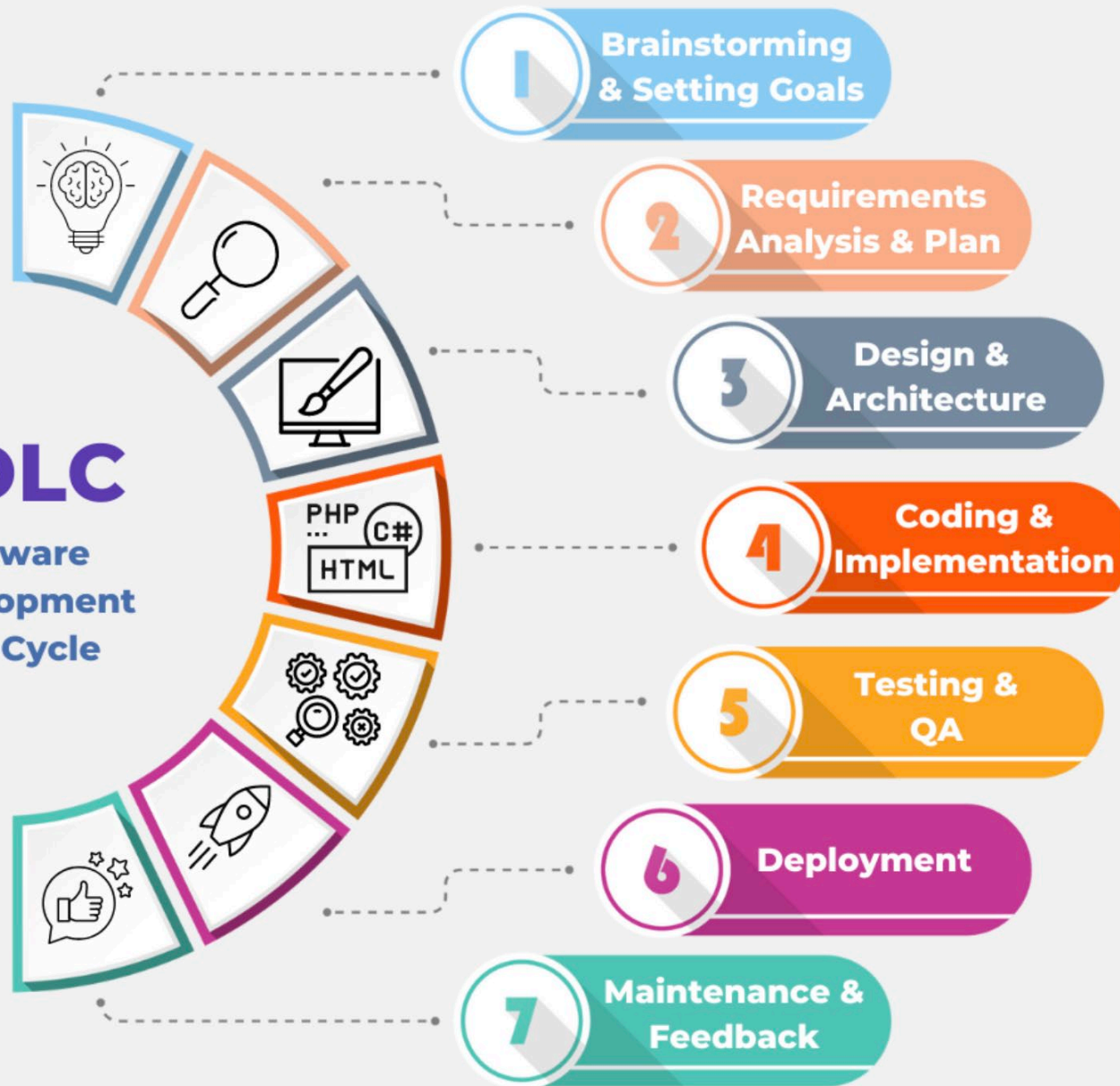


- Working closely together with (embedded into) engineering teams
- Driving initiatives towards secure software development lifecycle
- Spider in the net: orchestrates requirements, initiatives etc. among business / engineering and other security departments

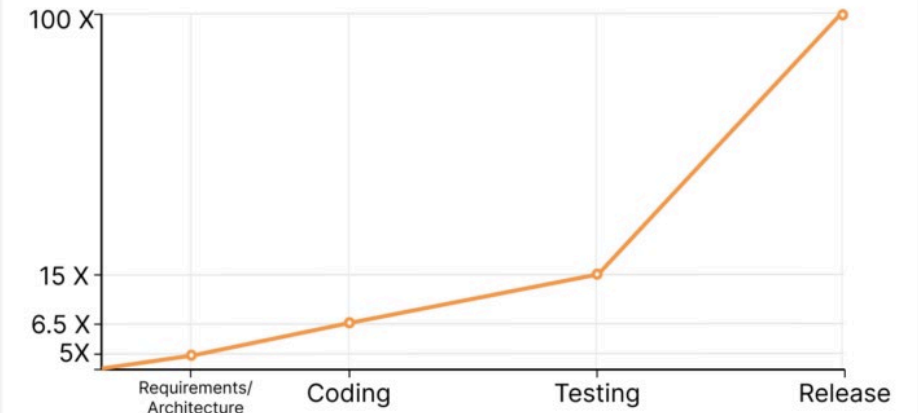


SDLC

Software
Development
Life Cycle



Relative cost of Bug Fixes
Based on Phase of SDLC





General Idea

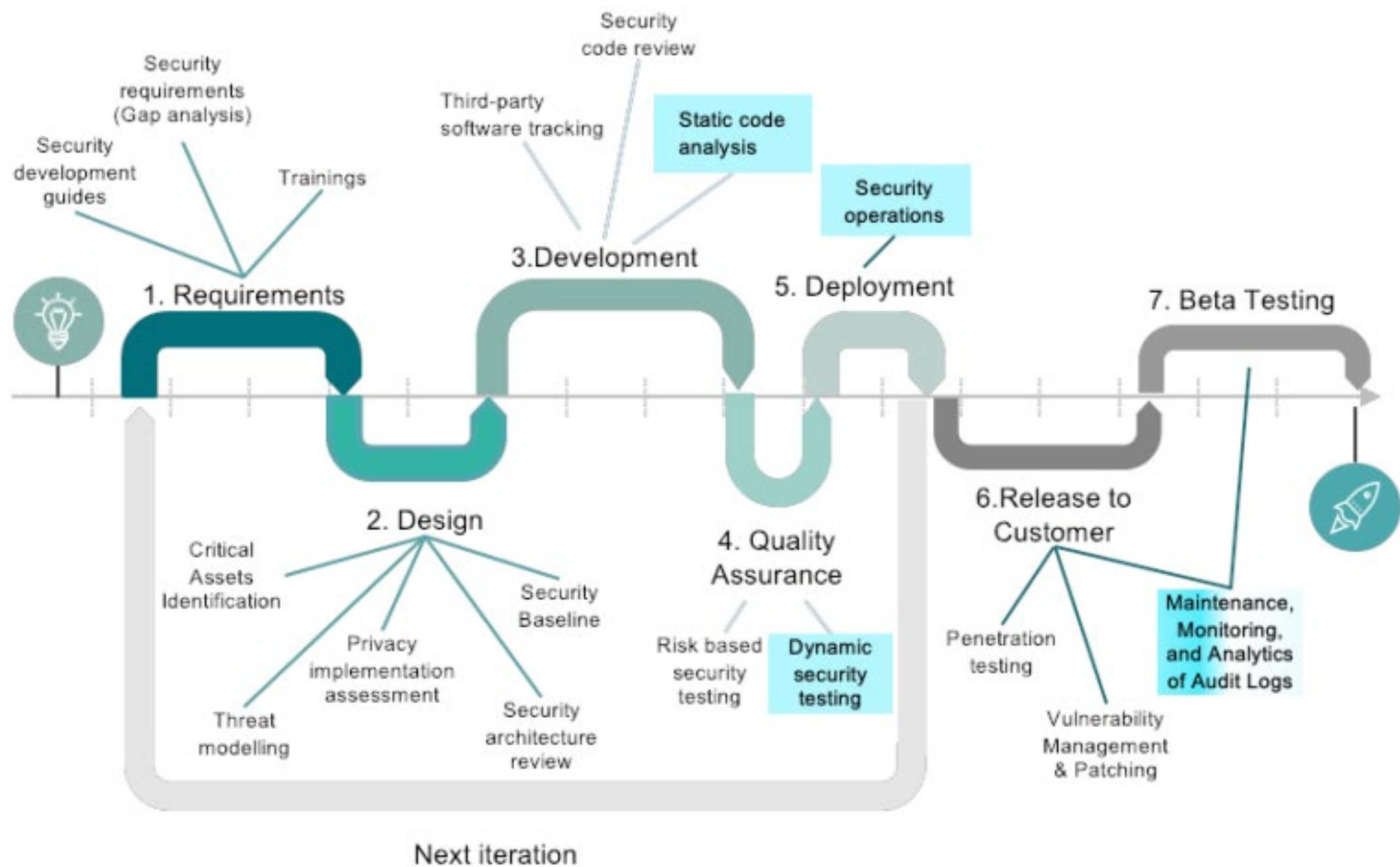
- Creating good software is hard
- To identify bugs as early as possible, requirements engineering is emphasised
- Same applies for adding security: **shift left** in the SDLC
- Goal is to create **safe and secure** software

Towards a SSDLC



Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law







Example: SDL at Microsoft

Phase	Practice
Training	Core Security Training
Requirements	Establish Security Requirements, Create Quality Gates/Bug Bars, Perform Security and Privacy Risk Assessments
Design	Establish Design Requirements, Perform Attack Surface Analysis/Reduction, Use Threat Modelling
Implementation	Use Approved Tools, Deprecate Unsafe Functions, Perform Static Analysis

Phase	Practice
Verification	Perform Dynamic Analysis, Perform Fuzz Testing, Conduct Attack Surface Review
Release	Create an Incident Response Plan, Conduct Final Security Review, Certify Release and Archive
Response	Execute Incident Response Plan



Threat Modelling

Stages of Process for Attack Simulation & Threat Analysis (PASTA)

1. Define Objectives

- Identify Business Objectives
- Identify Security and Compliance Requirements
- Business Impact Analysis

2. Define Technical Scope

- Capture the Boundaries of the Technical Environment
- Capture Infrastructure | Application | Software Dependencies

3. Application Decomposition

- Identify Use Cases | Define App. Entry Points & Trust Levels
- Identify Actors | Assets | Services | Roles | Data Sources
- Data Flow Diagramming (DFDs) | Trust Boundaries

4. Threat Analysis

- Probabilistic Attack Scenarios Analysis
- Regression Analysis on Security Events
- Threat Intelligence Correlation and Analytics

5. Vulnerability & Weaknesses Analysis

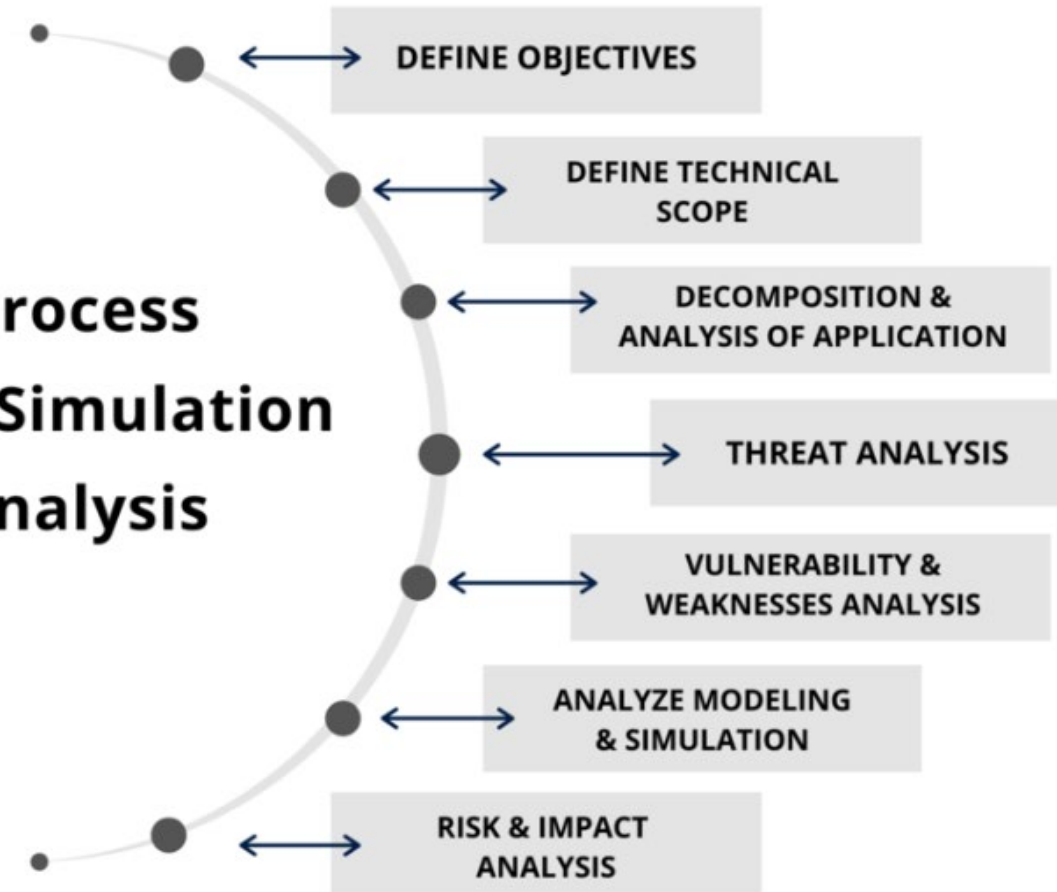
- Queries of Existing Vulnerability Reports & Issues Tracking
- Threat to Existing Vulnerability Mapping Using Threat Trees
- Design Flaw Analysis Using Use and Abuse Cases
- Scorings (CVSS/CWSS) | Enumerations (CWE/CVE)

6. Attack Modeling

- Attack Surface Analysis
- Attack Tree Development | Attack Library Mgt.
- Attack to Vulnerability & Exploit Analysis Using Attack Trees

7. Risk & Impact Analysis

- Qualify & Quantify Business Impact
- Countermeasure Identification and Residual Risk Analysis
- ID Risk Mitigation Strategies





Threat Modelling at Microsoft

S T R I D E



SPOOFING

In the context of information security, and especially network security, a spoofing attack is a situation in which a person or program successfully identifies as another by falsifying data, to gain an illegitimate advantage.



TAMPERING

Tampering can refer to many forms of sabotage but the term is often used to mean intentional modification of products in a way that would make them harmful to the consumer.



REPUDIATION

In digital security, non-repudiation means a service that provides proof of the integrity and origin of data, or an authentication that can be said to be genuine with high confidence.



INFO DISCLOSURE

Information disclosure is the unwanted dissemination of data, technology, or privacy. legal and political issues surrounding them. It is a violation of data privacy[2] or data protection. The challenge of data privacy is to use data ²



DENIAL OF SERVICE

A denial-of-service attack (DoS attack) is a cyber-attack in which the perpetrator seeks to make a machine or network resource unavailable to its intended users by temporarily or indefinitely disrupting services of a host connected to the ²

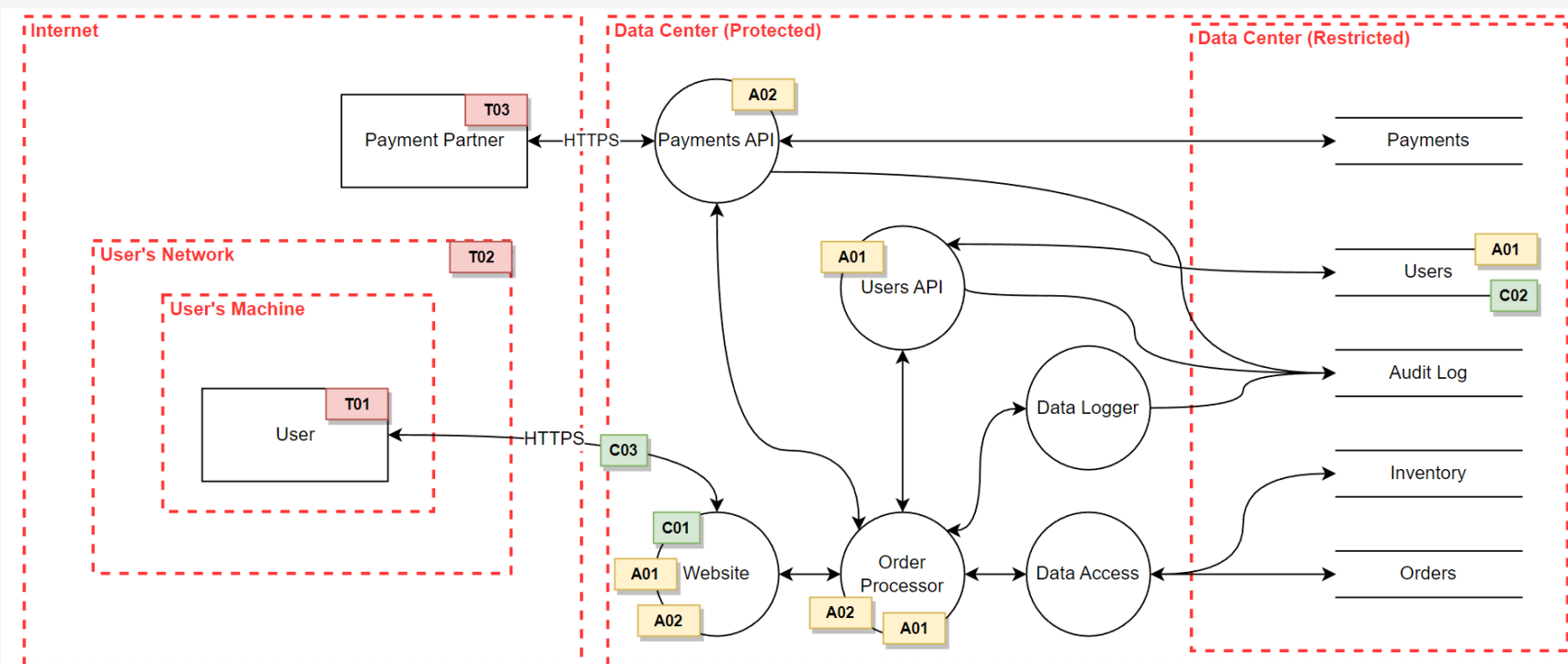


ELEVATION OF PRIVILEGE

Privilege escalation is the act of exploiting a bug, design flaw or configuration oversight in an operating system or software application to gain elevated access to resources that are normally protected from an application or user.



Threat Modelling



CYBER THREAT ACTOR

CYBER THREAT ACTOR		MOTIVATION
NATION-STATES		GEOPOLITICAL
CYBERCRIMINALS		PROFIT
HACKTIVISTS		IDEOLOGICAL
TERRORIST GROUPS		IDEOLOGICAL VIOLENCE
THRILL-SEEKERS		SATISFACTION
INSIDER THREATS		DISCONTENT

Assets	
ID	Description
A01	User Credentials
A02	Credit card data

Security Controls	
ID	Description
C01	Authentication
C02	Password hashing
C03	TLS

Threat Actors	
ID	Description
T01	Malicious user
T02	Man-in-the-middle
T03	Compromised payment partner

Secure Design Principles



Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Minimize Attack Surface Area	Don't trust Services
Establish Secure Defaults	Separation of Duties
Principle of Least Privilege	Avoid Security by Obscurity
Principle of Defense in Depth	Keep Security simple
Fail securely	Fix Security Issues correctly



Secure Design Principles

Minimize Attack Surface Area

- Every **feature** that is added to an application **adds a certain amount of risk** to the overall application
- The aim for secure development is to **reduce** the overall risk by reducing the **attack surface area**

Establish Secure Defaults

- The "**out-of-box**" experience for the user **should be secure**
- It should be up to the user to reduce their security - if they are allowed



Secure Design Principles

Principle of Least Privilege

- Accounts have the **least amount of privilege required** to perform their business processes
- This encompasses user rights and resource permissions, e.g.
 - CPU limits
 - memory
 - network
 - file system

Principle of Defense in Depth

- Where one control would be reasonable, **more controls that approach risks in different fashions** are better
- In-depth-controls can make severe vulnerabilities extraordinarily difficult to exploit



Secure Design Principles

Fail securely

- Whenever a transaction fails or code execution throws an exception it should always **"fail closed"** and never "fail open"

Don't trust Services

- Third party partners more than likely have differing security policies and posture
- Implicit **trust of externally run systems is not warranted**
- All external systems should be treated in a similar fashion

Separation of Duties

- Separation of duties is a key **fraud control**
- **Administrators should not also be users** of an application they are responsible for



Secure Design Principles

Avoid Security by Obscurity

- Security through obscurity is a weak security control, and nearly always fails when it is the only control
- The **security** of key systems **should not be reliant upon keeping details hidden**

Keep Security simple

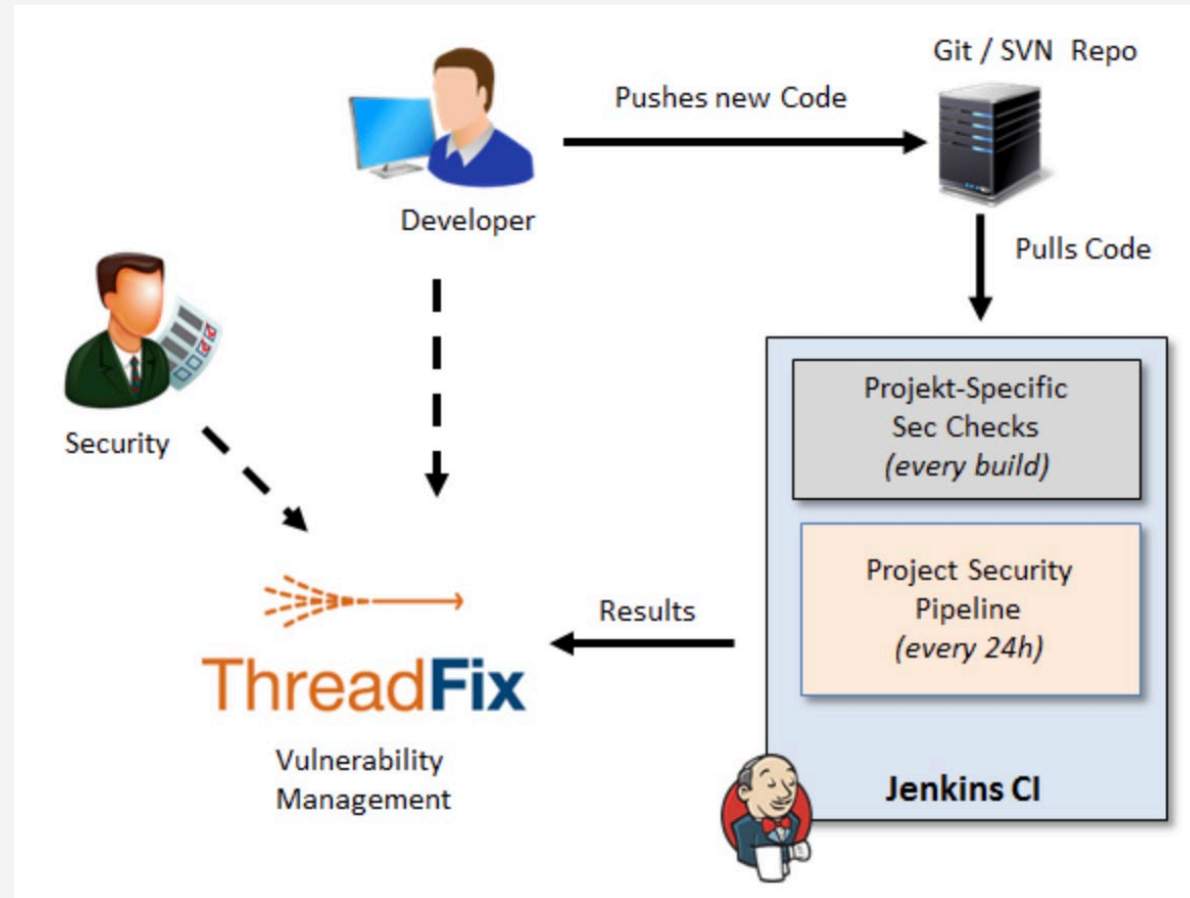
- **Attack surface and simplicity go hand in hand**
- Prefer straightforward and simple code over complex and over-engineered approaches
- Avoid the use of double negatives and complex architectures when a simpler approach would be faster and simpler



Fix Security Issues correctly

- Once a security issue has been identified, it is important to develop a test for it, and to **understand the root cause** of the issue
- It is likely that the security issue is widespread amongst all code bases, so **developing the right fix without introducing regressions** is essential

AppSec Pipelines





Static Application Security Testing (SAST)

Static application security testing (SAST) is a set of technologies designed to analyze application source code, byte code and binaries for coding and design conditions that are indicative of security vulnerabilities. SAST solutions analyze an application from the "inside out" in a nonrunning state. [⁶]

Dynamic Application Security Testing (DAST)

Dynamic application security testing (DAST) technologies are designed to detect conditions indicative of a security vulnerability in an application in its running state. Most DAST solutions test only the exposed HTTP and HTML interfaces of Web-enabled applications; however, some solutions are designed specifically for non-Web protocol and data malformation (for example, remote procedure call, Session Initiation Protocol [SIP] and so on). [⁷]



Static Security Tests

Strengths of SAST

- Scales well - can be run on lots of software, and can be run repeatedly (as with nightly builds or continuous integration)
- Useful for things that such tools can automatically find with high confidence, such as buffer overflows, SQL Injection Flaws, and so forth
- Output is good for developers - highlights the precise source files, line numbers, and even subsections of lines that are affected

Weaknesses of SAST

- Many types of security vulnerabilities are very difficult to find automatically (e.g. authentication problems, access control issues, insecure use of cryptography, etc.)
- Cover only a relatively small percentage of application security flaws
- High numbers of false positives
- Cannot find configuration issues not represented in the code
- Difficult to *prove* that an identified security issue is an actual vulnerability

Open Source SAST Tools (Examples)

	Tool	Supported Language
	SonarQube	20+ languages
	FindSecBugs plugin for SpotBugs	Java, Android, Groovy, Scala
	Bandit	Python
	Brakeman	Ruby on Rails



Open Source DAST Tools (Examples)

	Tool	Proxy	Scanner
	Zed Attack Proxy	✓	✓
 <i>w3af</i>	w3af	✗	✓
 Wapiti	Wapiti	✗	✓
	Nikto	✗	✓



Security Logging and Monitoring

Security Logging and Monitoring Failures

- Exploitation of insufficient logging and monitoring is the bedrock of nearly every major incident
- **Attackers** rely on the lack of monitoring and timely response to **achieve their goals without being detected**
 - Most successful attacks start with vulnerability probing
 - Allowing such probes to continue can raise the likelihood of successful exploit to nearly 100%



Security Logging and Monitoring

Examples of Insufficiencies

- Auditable events, such as logins, failed logins, and high-value transactions are not logged
- Warnings and errors generate no, inadequate, or unclear log messages
- Logs of applications and APIs are not monitored for suspicious activity
- Logs are only stored locally
- Appropriate alerting thresholds and response escalation processes are not in place or effective
- Penetration testing and scans by automated tools do not trigger alerts
- The application is unable to detect, escalate, or alert for active attacks in real time or near real time



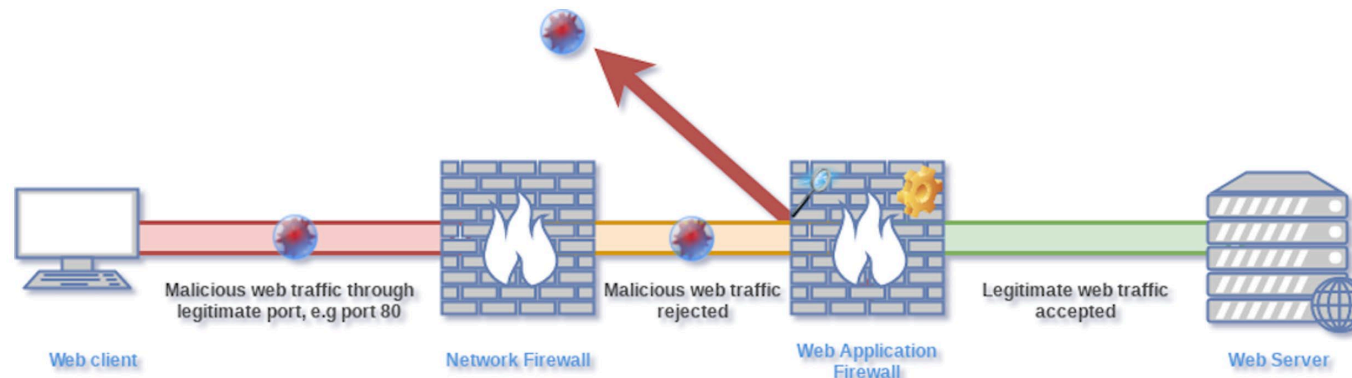
Some Aspects for Prevention

Prevention

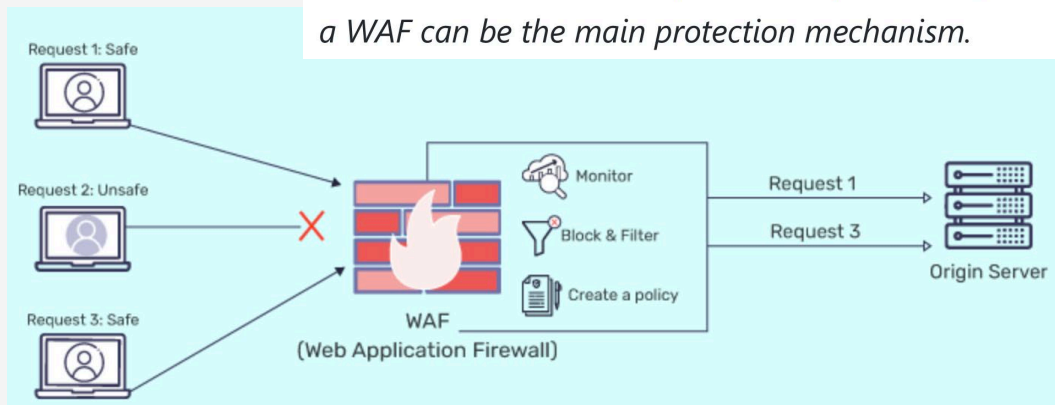
- Ensure all login, access control failures, and server-side input validation **failures can be**
 - **logged with sufficient user context** to identify suspicious or malicious accounts
 - **held for sufficient time** to allow delayed forensic analysis
- Ensure that logs are generated in a format that can be easily consumed by a **centralized log management solution**
 - e.g. [Elastic Stack](#) (Kibana, Elasticsearch, Logstash & Beats)
- Ensure **high-value transactions have an audit trail** with integrity controls to prevent tampering or deletion
 - e.g. append-only database tables or similar
- Establish effective monitoring and alerting such that **suspicious activities are detected** and responded to **in a timely fashion**
- Establish or adopt an incident response and recovery plan



Web Application Firewalls



💡 *An application should be able to protect itself! Use a WAF only as a secondary defense mechanism to achieve **Defense in Depth**! For legacy systems (with no feasible way to patch directly) a WAF can be the main protection mechanism.*



Source: <https://www.indusface.com/blog/how-web-application-firewall-works/>

A web application firewall (WAF) is an application firewall for HTTP applications. It applies a set of rules to an HTTP conversation. Generally, these rules cover common attacks such as cross-site scripting (XSS) and SQL injection.

While proxies generally protect clients, WAFs protect servers. A WAF is deployed to protect a specific web application or set of web applications. A WAF can be considered a reverse proxy. WAFs may come in the form of an appliance, server plugin, or filter, and may be customized to an application. The effort to perform this customization can be significant and needs to be maintained as the application is modified. [^8]

Risk in the use of WAFs

- "Yet-another-proxy" (increased complexity of the IT infrastructure)
- Organisational tasks
- Training the WAF on each new release of the web application
- Testing
- False positives (which may have a significant business impact)
- More complex troubleshooting
- WAFs also have/generate errors
- Responsibility for system-wide error situations
- Cost-effectiveness

WAF Modes

- **Blocking Mode:** Normal operational mode where the WAF blocks requests it identified as malicious.
- **Monitoring Mode:** The WAF logs alerts but does not block the corresponding requests.
- **Learning Mode:** The WAF learns from good traffic (e.g. by allowlisted IPs) what the normal use cases and input are.

💡 *Learning Mode might lead to false positives on new application releases when the WAF did not learn any traffic for new functionality.*



Security Champions

Security Champions are "active members of a team that may help to make decisions about when to engage the Security Team". They act as a core element of security assurance process within the product or service, and hold the role of the Single Point of Contact (SPOC) within the team.

It's all a question of scalability...

Developers will outnumber Security staff in almost every organization. There is *no way* to scale this up without either heavy automation or offloading security responsibility to the development teams!



Security Operations Center

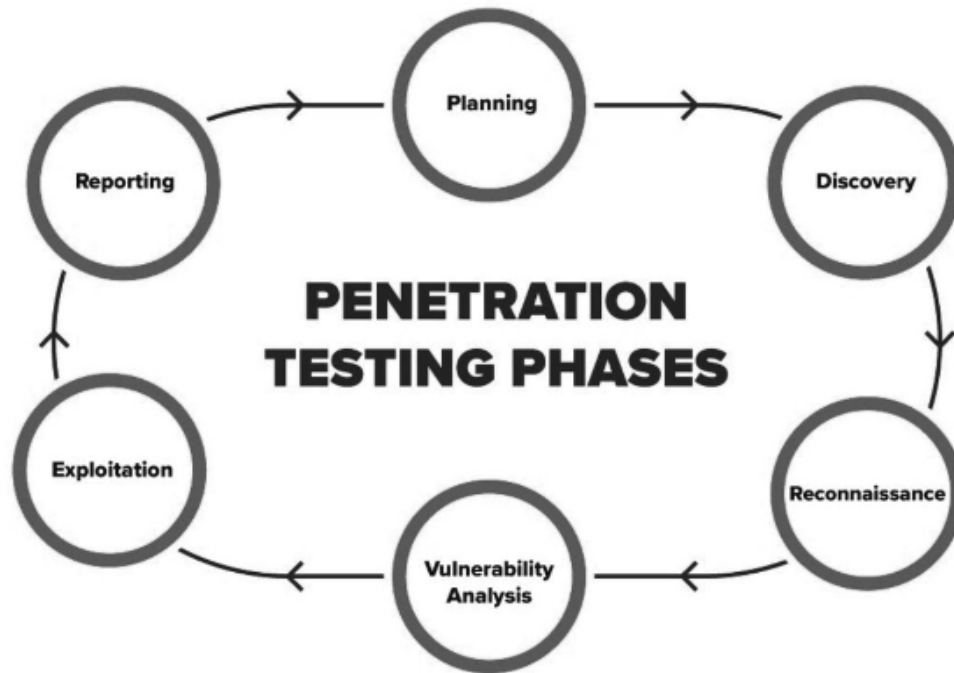


- Main task: stop the bleeding
- Also responsible for vulnerability management and engineering around SIEM systems
- Blue team and sometimes red team





Red Teaming and Pentests



Microsoft Bounty Programs

July 01, 2020 to June 30, 2021

\$13.6M

in bounty rewards



17

Bounty programs



1,261

Eligible vulnerability reports



341

Researchers awarded



\$200K

Biggest reward